

BlackBox Testig

Usecase testing

Introduction

- Previously, Input variables with their ranges and boundaries, business rules as represented in decision tables, and system behaviors as represented in state-transition diagrams.
- Now it is time to consider test cases that exercise a system's functionalities from start to finish by testing each of its individual transactions.
- Defining the transactions that a system processes is a vital part of the requirements definition process.
- Various approaches to documenting these transactions have been used in the past. Examples include flowcharts, HIPO diagrams, and text.
- Today, the most popular approach is the use case diagram.

Introduction

- Like decision tables and state-transition diagrams, use cases are usually created by developers for developers.
- But, like these other techniques, use cases hold a wealth of information useful to testers.
- Use cases were created by Ivar Jacobsen and popularized in his book *Object-Oriented Software Engineering: A Use Case Driven Approach*. Jacobsen defines a "use case" as a scenario that describes the use of a system by an actor to accomplish a specific goal.

Usecase

- By "actor" we mean a user, playing a role with respect to the system, seeking to use the system to accomplish something worthwhile within a particular context.
- Actors are generally people although other systems may also be actors.
- A "scenario" is a sequence of steps that describe the interactions between the actor and the system.
- Note that the use case is defined from the perspective of the user, not the system.
- The set of use cases makes up the functional requirements of a system.

Usecase

- The Unified Modeling Language notion for use cases is:

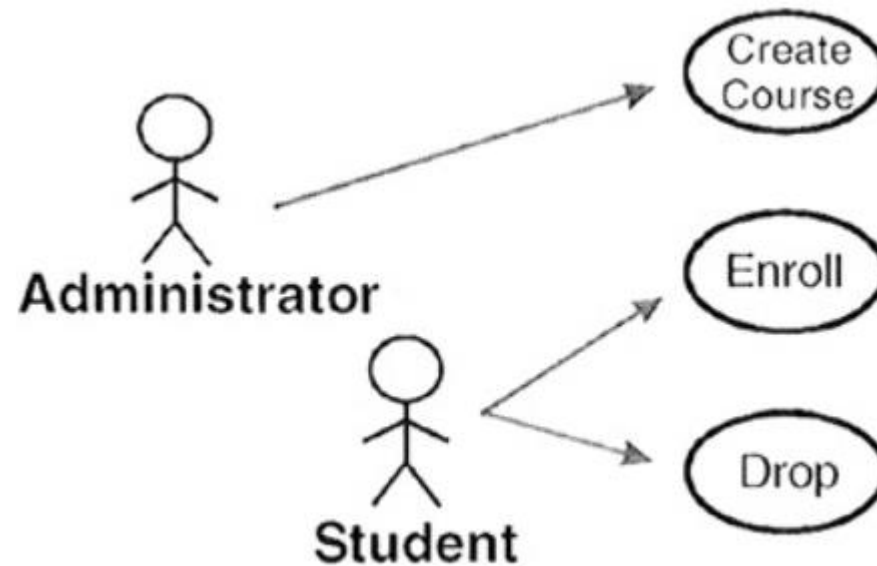


Figure 9-1: Some Stateless University use cases.

- The stick figures represent the actors, the ellipses represent the use cases, and the arrows show which actors initiate which use cases.

Usecase

- The value of use cases is that they:
- Capture the system's functional requirements from the user's perspective; not from a technical perspective, and irrespective of the development paradigm to be used.
- Can be used to actively involve users in the requirements gathering and definition process.
- Provide the basis for identifying a system's key internal components, structures, databases, and relationships.
- Serve as the foundation for developing test cases at the system and acceptance level.

Usecase Template

Usecase Example

Test case

- To test the implementation, the basic rule is to create at least one test case for the main success scenario and at least one test case for each extension.
- Because use cases do not specify input data, the tester must select it. Typically we use the equivalence class and boundary value techniques described earlier
- To create test cases, start with normal data for the most often used transactions. Then move to boundary values and invalid data.
- Next, choose transactions that, while not used often, are vital to the success of the system (i.e., **Shut Down The Nuclear Reactor**).

Test case

- Try transactions in strange orders. Violate the preconditions (if that can happen in actual use).
- If a transaction has loops, don't just loop through once or twice—be diabolical.
 - Look for the longest, most convoluted path through the transaction and try it.
- If transactions should be executed in some logical order, try a different order. Instead of entering data top-down, try bottom-up.
- Create "goofy" test cases. If you don't try strange things, you know the users will.

Test case

- Most paths through a transaction are easy to create.
- They correspond to valid and invalid data being entered.
- More difficult are those paths due to some kind of exceptional condition—low memory, disk full, connection lost, driver not loaded, etc.
 - It can be very time consuming for the tester to create or simulate these conditions
 - Fortunately, a tool is available to help the tester simulate these problems—
 - Holodeck, created by James Whittaker and his associates at Florida Institute of Technology. Holodeck monitors the interactions between an application and its operating system. It logs each system call and enables the tester to simulate a failure of any call at will. In this way, the disk can be "made full," network connections can "become disconnected," data transmission can "be garbled," and a host of other problems can be simulated.

Application and Limitations

- Transaction testing is generally the cornerstone of system and acceptance testing.
- It should be used whenever system transactions are well defined
- While creating at least one test case for the main success scenario and at least one for each extension provides some level of test coverage
 - it is clear that, no matter how much we try, most input combinations will remain untested.
- Do not be overconfident about the quality of the system at this point.